VULNERABILITY ASSESSMENT REPORT

Passive Reconnaissance

Prepared by	Future Interns - Cyber Security Task 1 (2026)
Report Date	02 April 2026
Target	https://demo.testfire.net
Assessment Type	Passive Vulnerability Assessment
Methodology	OWASP Testing Guide
Tools Used	Nmap, Browser DevTools, Gobuster, OWASP ZAP
Classification	Confidntial - for Training purposes
Author	AMEGANDJI Akuete Anoumou Antoine

1. Executive Summary

A passive security assessment was conducted on the public website demo.testfire.net, using safe, non-intrusive techniques. The purpose of this audit is to identify security misconfigurations, missing security headers, outdated components, and weaknesses that could increase the risk of attacks such as clickjacking, content injection, or information leakage.

- Total Vulnerabilities Identified: 5
- High Severity: 2 | Medium Severity: 2 | Low Severity: 1
- Immediate remediation required for all HIGH severity findings

HIGH	MEDIUM	LOW
2 Findings	2 Findings	1 Finding

2. Scope & Methodology

a- Assessment Target

Target Name	AltoroMutual Demo Banking Application
Target URL	http://demo.testfire.net
IP Address	65.61.137.117

Open Ports	80 (HTTP), 443 (HTTPS), 8080 (HTTP Alt)
Assessment Mode	Passive

b- Methodology

Phase 1 — Recon	Passive fingerprinting: server headers, technology stack, cookies
Phase 2 — Enumeration	Page and directory enumeration via documentation and DevTools
Phase 3 — Analysis	Inspection of login forms, HTTP headers, and session handling
Phase 4 — Classification	Map findings to CWE/OWASP and assign CVSS 3.1 scores
Phase 5 — Reporting	Document in business language with clear remediation guidance

3. Reconnaissance Findings

Web Server	Apache Tomcat / IBM WebSphere (via Server response header)
Framework	Java EE / JSP (JavaServer Pages)
TLS Version	TLS 1.0 detected — outdated, known vulnerabilities
Cookies	Session cookie without Secure and HttpOnly flags observed
Error Pages	Java stack traces visible in error responses

4. Vulnerability Findings Summary

Vulnerability	Severity	CVSS	Affected Component
SQL Injection (Login)	HIGH	9.8 / 10	/login.jjsp — username field
Cross-Site Scripting (XSS)	HIGH	8.8 / 10	/search.aspx — query parameter
Missing Security Headers	MEDIUM	6.1 / 10	All pages — HTTP response headers
Weak Session Management	MEDIUM	5.4 / 10	Session cookie configuration
Verbose Error Messages	LOW	3.7 / 10	Error pages — stack traces exposed

5. Detailed Findings

VULN-01 · SQL Injection on Login Page CWE-89: SQL Injection	■ HIGH CVSS 9.8
Endpoint	/login.aspx — username and password input fields
Evidence	Login form does not sanitize input; classic SQLi patterns applicable
Business Impact	Complete database compromise; unauthorized admin access; data breach
Remediation	Use parameterized queries / prepared statements. Never concatenate user input into SQL.

VULN-02 · Reflected Cross-Site Scripting (XSS) CWE-79: Cross-Site Scripting	■ HIGH CVSS 8.8
Endpoint	/search.aspx?query= — reflected in HTML output without encoding
Evidence	Search query appears verbatim in the rendered HTML response
Business Impact	Session hijacking, credential theft, phishing via crafted URL
Remediation	Encode all user-supplied input before HTML rendering. Implement Content-Security-Policy header.

VULN-03 · Missing HTTP Security Headers CWE-693: Protection Mechanism Failure	■ MEDIUM CVSS 6.1
Affected Scope	All HTTP responses — missing: X-Frame-Options, CSP, HSTS, X-Content-Type-Options
Business Impact	Clickjacking, MIME sniffing exploits, Man-in-the-Middle attacks on HTTP
Remediation	Add security headers via server middleware. Enforce HTTPS with HSTS header.

VULN-04 · Weak Session Cookie Configuration CWE-614: Cookie without Secure Flag	■ MEDIUM CVSS 5.4
Affected Component	JSESSIONID session cookie — missing Secure and HttpOnly flags
Business Impact	Session cookie stolen via HTTP or JavaScript (XSS pivot)
Remediation	Set Secure=true, HttpOnly=true, SameSite=Strict on all session cookies.

VULN-05 · Verbose Error Messages CWE-209: Error Message Information Exposure	■ LOW CVSS 3.7
Affected Component	Error pages — Java stack traces visible to the end user
Business Impact	Reveals server paths, framework versions, and code structure to attackers
Remediation	Implement generic error pages. Log details server-side only.

6. Recommendations

a- 1Immediate Actions (HIGH)

Implement parameterized queries to eliminate SQL Injection.

Encode all user input in HTML output to prevent XSS attacks.

b- Short-Term (MEDIUM)

Add all required security response headers (CSP, HSTS, X-Frame-Options).

Set Secure, HttpOnly, and SameSite=Strict on all session cookies.

c- Long-Term (LOW)

Upgrade TLS 1.0 to TLS 1.3. Implement a Web Application Firewall (WAF).

Establish monthly vulnerability scanning cadence.

7. Conclusion

Our passive security assessment uncovered five vulnerabilities, highlighted by two high-severity findings: SQL Injection and Cross-Site Scripting (XSS). These represent some of the most significant risks to modern web applications. This review reflects a rigorous, ethical approach, providing actionable remediation strategies tailored for business stakeholders.